# EE5203 L02 Practice Worksheet

Values, decisions, loops, and arrays

Basri Erdogan

**Expected time:** 30-45 minutes **Policy:** Attempt every question before consulting the worked solutions. Answer on paper first; the debugger is for verifying, not discovering.

## Part A - Values and arithmetic

1. Trace each program by hand and give every variable's final value.

```
/* (a) */                    /* (b) */
int x = 4;                   int a = 14;
int y = 2;                   int b = 5;

x = x + 3;                   int q = a / b;
y *= 5;                      int r = a % b;
x++;                         a = q * b + r;
```

   For (b), state in one sentence the general relationship your final value of `a` demonstrates.

2. All operands are `int`. Evaluate:

   a. `7 / 2`
   b. `7 % 2`
   c. `2 / 7`
   d. `(19140 / 8) * 8` — why is the result not `19140`?

3. A student computes an average as `sum / count` and is surprised the result ignores the fraction. Without changing types, how could the student check whether the true average was closer to the next integer? (Hint: `%`.)

## Part B - Decisions

4. For each value of `sample`, give the final `status_code`:

```
if (sample < 1500)        status_code = -1;
else if (sample > 3000)   status_code = 1;
else                      status_code = 0;
```

   a. `sample = 1499`
   b. `sample = 1500`
   c. `sample = 3000`
   d. `sample = 3001`

5. This code compiles but is wrong. Explain what it actually does, what the programmer intended, and the one-character fix.

```
if (mode = 2) {
    fan_on = 1;
}
```

6. `int v = 3000;` — evaluate each expression as `1` (true) or `0` (false):

   a. `v >= 0 && v <= 4095`
   b. `v < 1500 || v > 3000`

c. `!(v > 2500)`

7. Rewrite this chain as a single `if/else` by combining conditions with a logical operator, preserving behavior exactly:

```
if (mode == 1) {
    heater_on = 1;
} else if (mode == 2) {
    heater_on = 1;
} else {
    heater_on = 0;
}
```

## Part C - Loops

8. Complete a trace table for this loop: one row per repetition, columns for `i` and `total` **after** the body.

```
int total = 0;

for (int i = 1; i <= 4; i++) {
    total += i * i;
}
```

   a. How many times does the body run?
   b. Final value of `total`?
   c. This loop uses `<=` correctly. Why is `<=` safe here but dangerous in `for (int i = 0; i <= 8; i++) { sum += samples[i]; }`?

9. Rewrite Question 8's loop as a `while` loop with identical behavior. Label which line plays the START, TEST, and UPDATE roles.

10. Each loop below has one defect. Name the defect and the observable symptom in the debugger.

```
/* (a) */
int count = 5;

while (count > 0) {
    led_state = !led_state;
}
```

```
/* (b) */
int sum;

for (int i = 0; i < 8; i++) {
    sum += samples[i];
}
```

11. A countdown must visit `5, 4, 3, 2, 1` and stop. Write the `for` loop, and state the value of the counter when the loop test finally fails.

## Part D - Arrays and the accumulator pattern

12. `c` `uint16_t values[4] = { 1200, 4300, 2600, 800 };`

   a. List the valid indices.
   b. Which element does `values[2]` select?
   c. What clamping (limit `4095`) changes, if anything?

13. Trace the full pipeline on the four-element data of Question 12 — clamp above `4095`, sum, count strictly above `2500`, average over 4:

| Result | Value |
|---|---|
| array after clamping | |
| `sum` | |
| `high_count` | |
| `average` | |

Then apply the L02 status rule (`< 1500` LOW, `> 3000` HIGH, else OK) to the average.

14. One loop, one pass: write a single `for` loop over `values[4]` that finds the **largest** element. State what starting value your `max` variable needs and why `0` works here but is not a safe general choice.

15. In the lab program, the clamp (`if (samples[i] > 4095) samples[i] = 4095;`) is placed **before** `sum += samples[i];` inside the same loop body. If the two statements were swapped, which final results change and by how much, for the lab's data set `{850, 1250, 4095, 4200, 2050, 900, 3100, 2800}`?

## Part E - Debugger evidence

16. You claim your loop visits all eight elements exactly once. Describe the breakpoint placement and the two variables you would watch to prove it, and what values you expect on the first and last pause.

17. A student's `average` is `0` although the samples are clearly nonzero. Give three distinct explanations consistent with a clean build, and one debugger observation that separates each from the others.

## Exit self-assessment

Mark one:

☐ Ready for the L02 lab without additional review
☐ Need to review integer arithmetic and `=` vs `==`
☐ Need to review loop tracing
☐ Need to review array indices and the accumulator pattern